1. ¿Cómo fue modelado el Warehouse?

El Warehouse se modeló mediante una combinación de objetos que representan los elementos físicos y operativos del almacén y un grafo que representa los caminos transitables.

La clase Warehouse, ubicada en domain/Warehouse.py, contiene las entidades del sistema: racks, pasillos, productos, pedidos, pickers, puertas y zonas de despacho.

Cada elemento tiene propiedades propias, como ubicación, dimensiones, capacidad o restricciones de movimiento, según corresponda.

Además, el almacén utiliza coordenadas espaciales para ubicar sus elementos y construir las conexiones que permiten el desplazamiento de los pickers.

Esto permite separar dos aspectos: la representación física y lógica del almacén, y la representación de los recorridos posibles.

2. ¿Qué representación de la estructura de datos utilizaron y por qué?

Para representar la circulación se utilizó un grafo dirigido y ponderado, implementado mediante listas de adyacencia.

Cada vértice representa una posición transitable y cada arista representa un desplazamiento permitido hacia otra posición. La dirección de las aristas permite respetar los sentidos de circulación de los pasillos.

El proyecto utiliza dos representaciones del grafo:

- Grafo completo: contiene las posiciones transitables individuales; se utiliza para el movimiento de los pickers.
- Grafo reducido: conserva puntos relevantes, como esquinas, accesos y conexiones, y representa trayectos mediante aristas ponderadas por su longitud.

Elegimos listas de adyacencia porque permiten recorrer eficientemente los vecinos de cada vértice y son apropiadas cuando una posición tiene pocas conexiones respecto de la cantidad total de posiciones del almacén.

3. ¿Qué algoritmos utilizaron para resolver los problemas encontrados?

Problema	Algoritmos utilizados
Agrupar productos espacialmente	K-Means con inicialización inspirada en K-Means++
Distribuir unidades entre pickers	Backtracking con podas y estrategia voraz
Seleccionar zonas de despacho	Búsqueda exhaustiva o estrategia voraz
Buscar caminos mínimos	Dijkstra y A*
Determinar el orden de visita	Orden original, cercanía y zonas

4. ¿Por qué eligieron dichos algoritmos?

Se eligieron buscando un equilibrio entre calidad de las soluciones, tiempo de ejecución y complejidad de implementación.

K-Means permite agrupar espacialmente los productos sin explorar todas las particiones posibles. K-Means++ mejora la selección inicial de los centroides.

En la asignación de pickers, el backtracking permite comparar exhaustivamente alternativas pequeñas, mientras que la estrategia voraz evita una exploración exponencial cuando aumenta el número de unidades.

Dijkstra garantiza caminos mínimos con pesos no negativos. A* utiliza una heurística Manhattan que, bajo las condiciones implementadas, conserva la optimalidad y puede explorar menos nodos.

Finalmente, las tres estrategias de recorrido permiten comparar un orden simple, una heurística basada en distancias reales y una organización geométrica por orientación de racks.

5. ¿Qué ocurre si todos los pasillos tienen el mismo largo?

Es importante diferenciar pasillos de igual longitud de aristas de igual peso.

En nuestro proyecto, el grafo completo utiliza desplazamientos entre celdas adyacentes con peso 1. En ese grafo, una búsqueda BFS podría encontrar caminos de distancia mínima medida en cantidad de pasos.

Sin embargo, el grafo reducido tiene aristas que representan trayectos de diferentes cantidades de celdas. Aunque los pasillos físicos tuvieran igual longitud, las aristas reducidas no necesariamente tendrían todas el mismo peso.

Por eso, Dijkstra y A* siguen siendo adecuados para el grafo reducido.

Si todas las aristas tuvieran exactamente el mismo peso, BFS sería una alternativa más sencilla para encontrar caminos mínimos.

6. ¿Cómo cambia el comportamiento del algoritmo cuando aumenta el tamaño del Warehouse?

Al aumentar el almacén pueden aumentar los vértices y aristas del grafo, las ubicaciones de productos, los pedidos, las waves y los viajes de los pickers.

Esto afecta a los distintos algoritmos de manera diferente.

La generación de waves incrementa su costo al procesar más registros, ubicaciones y clústers. La asignación exacta puede crecer exponencialmente con las unidades de una ronda, motivo por el cual se utiliza una alternativa voraz para cargas mayores.

En la búsqueda de caminos, el aumento del grafo implica potencialmente más nodos por explorar.

Según el documento de experimentación de búsqueda de caminos:

Escala	**Nodos expandidos por búsqueda: A***	**Dijkstra**
Básico	20,3	64,1
Mediano	24,7	95,6
Grande	72,6	423,7

En esos experimentos, A* exploró considerablemente menos nodos que Dijkstra, y la diferencia aumentó con el tamaño del almacén.

7. ¿Cómo determinaron la distancia entre las ubicaciones de una wave?

En el proyecto utilizamos dos criterios de distancia diferentes, según la etapa.

Durante la generación de waves, K-Means calcula la distancia euclídea cuadrada entre las ubicaciones de los productos y los centroides:

$$D^2(p,c) = (xp - xc)^2 + (yp - yc)^2 + (zp - zc)^2$$

Esta distancia sirve para agrupar registros espacialmente, pero no contempla obstáculos ni sentidos de circulación.

En cambio, al planificar los recorridos de los pickers, se utilizan las distancias mínimas calculadas sobre el grafo mediante Dijkstra o A*. Esas distancias sí consideran los caminos permitidos dentro del almacén.

Por lo tanto, la generación utiliza una aproximación geométrica, mientras que la planificación de viajes trabaja con las distancias de desplazamiento modeladas en el grafo.

8. ¿Cómo decidieron qué pedidos pertenecen a una misma wave?

La decisión se realiza en varias etapas:

1. Preparación: se consolidan los ítems de cada pedido y se determina la cantidad disponible según el stock y las prioridades.
2. Cálculo de K: se estima la cantidad inicial de clústers según las unidades disponibles, los pedidos distintos y las capacidades.
3. Agrupamiento: K-Means reúne los registros con ubicaciones espacialmente similares.
4. Construcción: cada clúster se divide en waves respetando las capacidades máximas de unidades y pedidos.

Un punto importante es que no agrupamos pedidos completos de forma indivisible. Agrupamos registros producto-pedido. Por eso, los productos de un mismo pedido pueden pertenecer a waves diferentes.

También mantenemos la restricción de que los registros procedentes de clústers diferentes no se mezclan en una misma wave.

9. ¿Qué estrategia utilizaron para ordenar las ubicaciones de una wave?

El sistema implementa tres estrategias para establecer el orden de visita de las ubicaciones asignadas a un picker.

Orden: visita las ubicaciones según el orden en que aparecen en la wave. No reorganiza los productos buscando reducir distancias.

Cercanía: desde la ubicación actual, selecciona la ubicación pendiente con menor distancia de recorrido. Repite hasta visitar todos los puntos y después se dirige al despacho.

Zonas: agrupa las ubicaciones de acuerdo con la orientación de sus racks. Ordena los racks verticales por coordenadas \((x,y)\) y los horizontales por \((y,x)\), visitando primero los verticales.

En la configuración predeterminada del asignador se utiliza cercanía, combinada con A* para los caminos.

La estrategia de ordenamiento determina la secuencia de visita; Dijkstra o A* calculan los caminos que conectan esas visitas.

10. ¿Qué diferencias existen entre las estrategias implementadas?

Estrategia	Criterio	Ventaja	Limitación
Orden	Secuencia original de registros	Sencilla y económica de calcular	No considera proximidad
Cercanía	Menor distancia desde la posición actual	Suele reducir el recorrido	Evalúa más alternativas
Zonas	Orientación y coordenadas de racks	Organiza espacialmente los puntos	No se adapta directamente a la posición del picker ni al sentido de los pasillos

Si m es la cantidad de ubicaciones y S el costo de una búsqueda de camino, sus costos teóricos son:

- Orden: $O(mS)$.
- Cercanía: $O(m^2S)$.
- Zonas: $O(m\log m+mS)$.

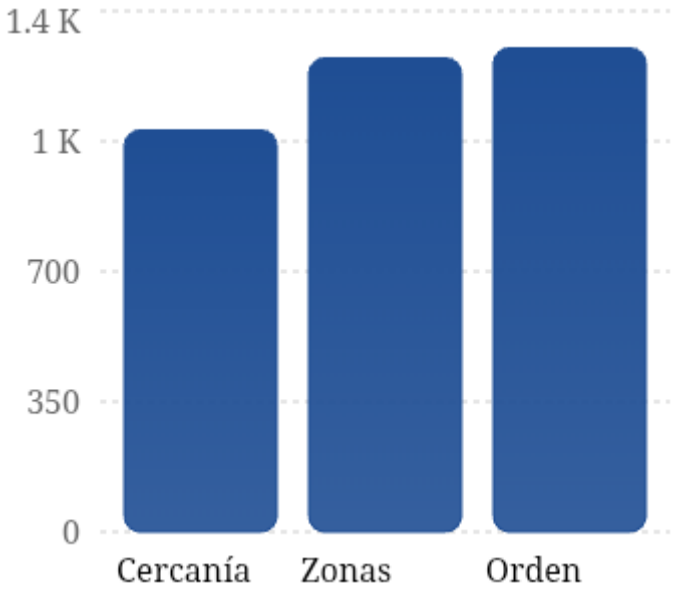
Cercanía es más costosa teóricamente, pero puede compensarlo al obtener mejores secuencias de visita.

11. ¿Cuál presenta menor distancia recorrida?

Según los resultados del documento de experimentación de búsqueda de caminos, Cercanía fue la estrategia con menor distancia promedio.

Distancia media recorrida por estrategia

Resultados de 39 escenarios del informe de búsqueda de caminos. Menor es mejor.



Cercanía superó o igualó a Orden en 37 de los 39 escenarios y obtuvo una distancia menor que Zonas en 33.

Esto respalda que cercanía es la mejor alternativa de las tres respecto de la distancia media en los escenarios evaluados, aunque no significa que encuentre siempre el recorrido óptimo.

12. ¿Cuál presenta menor tiempo de ejecución?

Es necesario distinguir el tiempo empleado en calcular una ruta del tiempo total de simulación.

El documento de experimentación informa los siguientes tiempos medios de simulación:

Estrategia	Tiempo medio
Cercanía	176 ms
Zonas	183 ms
Orden	194 ms

En estos experimentos, Cercanía también obtuvo el menor tiempo medio de simulación.

Aunque puede requerir más evaluaciones durante la planificación, produce recorridos más cortos y, por tanto, puede reducir la cantidad de ticks necesarios para completar las tareas.

Esto no implica que Cercanía tenga siempre el menor tiempo de cálculo aislado de la ruta.

En cuanto a los algoritmos de búsqueda de caminos, el documento también informa menores tiempos de simulación con A* que con Dijkstra en las tres escalas evaluadas.

13. ¿Existe una estrategia que sea mejor en todos los escenarios?

No se demostró que exista una estrategia que sea mejor en todos los escenarios y para todos los criterios.

Cercanía obtuvo los mejores resultados generales de distancia y tiempo medio de simulación, pero no ganó en absolutamente todos los casos.

Por ejemplo, en escenarios con waves muy pequeñas, las tres estrategias pueden producir los mismos recorridos porque existen muy pocas ubicaciones que ordenar.

En otros diseños de almacén, un orden geométrico puede resultar favorable.

Por lo tanto, la elección depende de las características del Warehouse, de los pedidos y de la métrica utilizada para evaluar el resultado.

14. ¿Qué trade-offs encontraron entre tiempo de ejecución y calidad del recorrido?

El principal compromiso aparece entre el costo de planificar un recorrido y la distancia que finalmente deben recorrer los pickers.

Cercanía realiza más comparaciones de destinos que Orden o Zonas. Sin embargo, al elegir destinos próximos, suele producir rutas de menor longitud.

Los experimentos mostraron que ese costo adicional no necesariamente aumenta el tiempo total de simulación: el ahorro de desplazamientos puede compensarlo.

También observamos un compromiso entre Dijkstra y A*. Ambos permiten obtener caminos mínimos bajo las condiciones del modelo, pero A* utiliza información heurística para orientar la búsqueda y generalmente expande menos nodos.

Finalmente, K-Means permite generar agrupaciones sin evaluar exhaustivamente todas las combinaciones, aunque no garantiza que los productos dentro de una wave tengan recorridos mínimos ni que todos estén próximos entre sí.

15. ¿Qué modificaciones realizarían para utilizar el sistema en un Warehouse real?

Para utilizar el sistema en un almacén real, propondríamos incorporar:

1. Distancias y tiempos reales de desplazamiento: considerar velocidades, giros, cruces, congestión y diferencias entre tipos de pasillo.
2. Obstáculos dinámicos: recalcular los caminos ante bloqueos temporales o cambios en la circulación.
3. Asignación dinámica de tareas: tener en cuenta la ubicación actual del picker, su carga restante y los cambios en la demanda.
4. Sincronización con inventario real: actualizar reservas, faltantes y reposiciones mediante un sistema de gestión de almacenes.
5. Criterios espaciales adicionales para las waves: evaluar distancias de recorrido o límites de dispersión entre productos, no solamente distancias euclídeas a los centroides.
6. Validación con datos operativos: comparar distancias estimadas, recorridos reales, tiempos de preparación, congestión y productividad antes de implementar el sistema en producción.

Estas modificaciones permitirían pasar de una simulación con restricciones conocidas a un sistema capaz de responder a condiciones operativas cambiantes.